



Day 4 — Quick OpenResty Integration (15–30 minutes)

Below is a **self-contained** OpenResty (Nginx + Lua) configuration that performs **non-blocking**, deferred ML scoring via your async sidecar, logs detections, and proxies traffic to your upstream app.

```
# nginx.conf (within http { ... })
lua_package_path "/usr/local/openresty/lualib/?lua;";

init_worker_by_lua_block {
    -- Preload modules once per worker
    local http = require "resty.http"
    local cJSON = require "cjson"
}

server {
    listen 80;
    server_name _;

    # Upstream app on port 8081 (Tomcat or your FastAPI)
    location / {
        access_by_lua_block {
            local http = require "resty.http"
            local cJSON = require "cjson"
            local seq

            -- 1. Build normalized sequence (mirror Python rules)
            local method = ngx.var.request_method
            local uri = ngx.var.request_uri
            -- Replace numeric IDs in path with <ID>
            local normalized = ngx.re.gsub(uri, "[0-9]+", "<ID>", "jo")
            seq = method .. " " .. normalized

            -- 2. Attach header (optional)
            ngx.req.set_header("X-ML-Sequence", seq)

            -- 3. Deferred ML scoring (non-blocking)
            ngx.timer.at(0, function(premature)
                if premature then
                    return
                end
                local httpc = http.new()
                httpc:set_timeout(2000)

                local res, err = httpc:request_uri("http://127.0.0.1:8080/score", {
                    method = "POST",
```

```

        body    = cJSON.encode({ seq = seq }),
        headers = { ["Content-Type"] = "application/json" }
    })

    if res and res.status == 200 then
        local body = cJSON.decode(res.body)
        if body.anomalous then
            -- 4. Log detections asynchronously
            local f = io.open("/var/log/waf_detections.log", "a+")
            if f then
                f:write(os.date("%Y/%m/%d %H:%M:%S") ..
                    " " .. ngx.var.remote_addr ..
                    " \"\" .. seq .. "\"\" ..
                    " score=" .. body.score .. "\n")
                f:close()
            end
            -- 5. (Optional) Call admin API to block IP:
            -- httpc:request_uri("http://127.0.0.1:9000/block", { ... })
        end
    end
end)
-- 6. Immediate pass-through
}

# Proxy to your upstream application
proxy_pass http://127.0.0.1:8081;
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}

# Static assets, health checks, etc.
location = /health {
    return 200 "OK";
}

}

# Upstream definition (if preferred instead of inline proxy_pass)
# upstream app {
#     server 127.0.0.1:8081;
# }

```

Setup & Run (5 minutes)

1. Install OpenResty (Ubuntu example):

```

sudo apt-get update
sudo apt-get install -y openresty

```

2. Place nginx.conf under /usr/local/openresty/nginx/conf/.

3. Create log file and set permissions:

```

sudo touch /var/log/waf_detections.log

```

```
sudo chown www-data:www-data /var/log/waf_detections.log
```

4. Reload OpenResty:

```
sudo openresty -s reload
```

How It Works

- **Immediate Pass-Through:** Requests are proxied with zero delay.
- **Deferred Scoring:** `ngx.timer.at(0,...)` calls ML sidecar asynchronously.
- **Detection Logging:** Anomalous sequences are appended to `/var/log/waf_detections.log`.
- **Optional Blocklisting:** You can extend the deferred function to call an admin/block API.

Total time: ~15 minutes to integrate OpenResty with your async ML sidecar for non-blocking anomaly detection.